

Conformal Prediction for Customer Churn Modeling on AWS

Problem Context and Variables

With the rapid growth of e-commerce platforms, understanding customer browsing behavior and purchase intentions has become essential for improving online sales performance. Companies increasingly rely on session-level big data to analyze how visitors interact with websites and what factors influence their purchasing decisions. This dataset contains 1 million simulated online shopping sessions. Each record represents a single user session and includes information about page visits, time spent on different page categories, traffic sources, visitor characteristics, and behavioral metrics collected during the browsing process.

Using this dataset, I intend to figure out whether a marketing team should send a discount coupon to a specific customer to encourage purchase.

Task: Probabilistic Binary Classification

Dataset: "Online Shoppers Purchasing Dataset" on Kaggle

(<https://www.kaggle.com/datasets/thuandao/online-shoppers-purchasing-dataset?resource=download>)

Administrative → Number of administrative pages visited during the session.

Administrative_Duration → Total time spent on administrative pages.

Informational → Number of informational pages visited during the session.

Informational_Duration → Total time spent on informational pages.

ProductRelated → Number of product-related pages visited during the session.

ProductRelated_Duration → Total time spent on product-related pages.

BounceRates → Percentage of visitors who leave the website after viewing only one page.

ExitRates → Percentage of pageviews that were the last in the session.

PageValues → Average value of pages visited before completing an e-commerce transaction.

SpecialDay → Indicator of proximity to special shopping days (e.g., Valentine's Day, Mother's Day).

Month → Month when the session occurred.

OperatingSystems → Operating system used by the visitor.

Browser → Browser used during the session.

Region → Geographic region of the visitor.

TrafficType → Source of the website traffic.

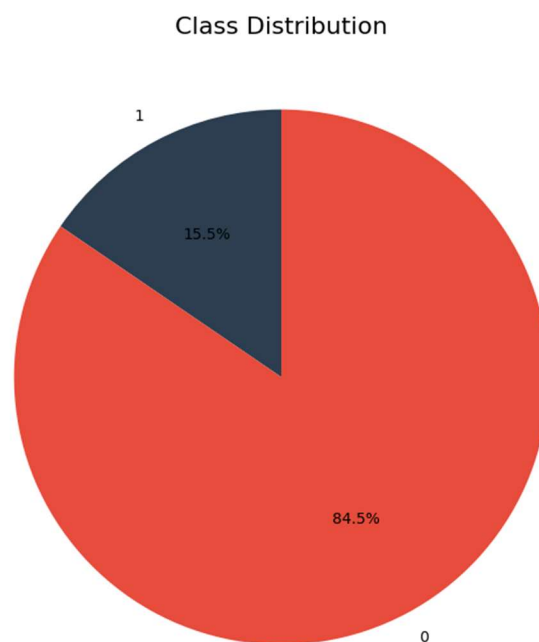
VisitorType → Type of visitor (New Visitor, Returning Visitor, Other).

Weekend → Boolean indicator showing whether the session occurred on a weekend.

Revenue → Boolean value indicating whether the session resulted in a purchase.

Data Selection, Preprocessing, Feature Engineering and Transformation

[1] Some features are of boolean and string data types. These have to be encoded as numbers to represent their categorical nature better. I encode these features accordingly. **[2]** Looking at the class distribution, It is observed that the dataset is highly imbalanced. This is something expected since most entries do not result in purchase:



[3] It is noticeable that genuine numeric features (non-categoricals) are not on the same scale. Although tree-based models do not typically require feature scaling, I will scale the numeric features to improve the baseline model (logistic regression) and to better handle extreme values. To this end, I am going to use RobustScaler, which handles extreme values efficiently

by removing the median and scaling based on IQR. Details of my models of choice will be discussed in the following sections in more detail.

[4] There are no missing values in the dataset. I do not need to implement imputation methods such as MissForest. Therefore, assumptions regarding missingness mechanisms such as MCAR or MAR are not relevant in this context.

[5] I am going to generate new features using mathematical interactions and proportions between the existing ones. Specifically:

- One possible interaction can be found between "ProductRelated_Duration" and "ProductRelated". By calculating "ProductRelated_Duration" / ("ProductRelated" + 1), we can simply calculate how much time the user spent on a single product page on average while also avoiding division by zero errors.

- The same logic can be applied to "Administrative_Duration" and "Administrative". Dividing the "Administrative_Duration" column by ("Administrative" + 1) will give us the average time spent on a single administrative page.

- Furthermore, "ProductRelated_Duration" / ("Administrative_Duration" + "Informational_Duration" + "ProductRelated_Duration" + 1) would give us the ratio of time the user spent inspecting products to the total time spent on the website.

- It is even possible to calculate the total page view count the user undertook by adding "Administrative" + "Informational" + "ProductRelated".

- For a more advanced approach, I also create cyclical features using the "Month" column. The calculations for this operation can be written as:

$$\begin{aligned}\text{Month_Sin} &= \sin\left(\frac{2\pi \cdot \text{Month}}{12}\right) \\ \text{Month_Cos} &= \cos\left(\frac{2\pi \cdot \text{Month}}{12}\right)\end{aligned}$$

[6] It is crucial to drop certain columns to prevent data leakage, eliminate noise, and reduce dimensionality.

- I am completely removing the "PageValues" column. Since this feature represents the average value of pages visited right before completing a transaction, including it would cause severe data leakage. It essentially feeds the model the final answer beforehand, which would result in artificially perfect scores during training but complete failure in a real scenario.

- I am also dropping "BounceRates" due to its massive 0.90 correlation with "ExitRates". Keeping both would lead to multicollinearity because they tell the exact same story about

users leaving the website. I chose to retain "ExitRates" because it is a slightly broader and more useful metric for the session's end.

- I am also removing the original raw duration and page count columns, such as "ProductRelated" and "ProductRelated_Duration". Because I have already combined these numbers to engineer much stronger features such as the average time spent per page and the focus ratio.

[7] The first critical step in splitting this dataset is doing so by preserving its imbalanced nature. To ensure the algorithm learns the true distribution of the data, I am going to implement a stratified splitting strategy. I will begin by isolating a 10% test set. To support conformal prediction (CP) and generate statistically guaranteed prediction intervals I need to create a dedicated calibration set from the remaining training data. I am going to allocate a small portion (0.05) of training data for calibration. By applying the same stratified logic, I am going to divide the data into training, calibration, and testing sets.

[8] Before moving to model training section, the features in the dataset should be transformed properly without introducing any data leakage. For categorical variables such as traffic and visitor types, I am going to avoid standard one-hot encoding. That approach would lead the feature space to blow like confetti that often confuse and slow down tree-based algorithms. Instead, I am going to implement target encoding strategy. By calculating the average purchase rate for each category strictly using only the training set, the algorithm learns to evaluate these categories based on their actual predictive potential rather than arbitrary labels. I will deliberately leave binary indicators, like the "Weekend" column, completely untouched because machine learning algorithms natively understand binary zero-and-one logic. For the continuous numerical features, I am going to apply a robust scaler. While it is true that tree-based algorithms do not benefit from such transformations, I am also planning to create an additional basic model like logistic regression (which benefits from standardization a lot) in order to compare the results with tree-based algorithms. Furthermore, RobustScaler excels at neutralizing extreme outliers. So as to guarantee mathematical isolation, the scaling operations and the target encoding maps will be learned exclusively from the training data and merely applied to the calibration and test sets.

Model Assumptions, Training, and Performance Metrics

[9] I am going to use some of the frontier algorithms used widely in industry, which are LightGBM, XGBoost, and additionally, Random Forest. I will also designate a Logistic Regression model as my baseline model. Because it is very fast and cheap to train, it perfectly answers the core business question: "Does the complexity and computational cost of advanced tree-based models actually yield a significant advantage over a basic mathematical approach?" I am going to use Optuna package for hyperparameter tuning. Instead of relying on blind Grid Search or Random Search, I am going to use the TPE (Tree-structured Parzen Estimator) sampler. TPE is

a Bayesian optimization algorithm that learns from the results of previous trials to intelligently guess the next best set of parameters. In terms of parameters, I will set a value of 200 for 'n_trials' because higher values give marginal returns. I will randomize the first 20 trials by setting 'n_startup_trials' to 20 in order to gather diverse baseline data before the Bayesian probability engine kicks in. However, I am going to use a much smaller experimentation set for Random Forest because it is computationally much more demanding than the other algorithms to train. I will use Optuna's Successive Halving Pruner to kill the trial if a specific hyperparameter set performs poorly in the first few folds. For the boosting models that learn iteratively (LightGBM and XGBoost/CatBoost), I am going to embed an early stopping mechanism. If a model fails to improve its log loss over a consecutive number of trees (epochs), it halts the training for that specific fold. AWS EMR will be used to leverage CPU resources.

[10] Perhaps the most critical architectural decision in this pipeline is the choice of the optimization metric. While many classification tasks default to ROC-AUC or Log Loss, I am going to minimize the Brier Score for this project. The reasoning is deeply tied to business utility. AUC purely measures ranking (does the model score a buyer higher than a non-buyer?), but it completely ignores the actual truth of the probabilities. Brier Score, essentially the mean squared error of probabilities, forces the models to be perfectly calibrated. In a real-world business scenario, if our model predicts an 80% probability of a customer making a purchase, we need that customer to actually purchase exactly 8 out of 10 times. By aggressively optimizing the Brier Score, I ensure that our models do not just separate classes, but output reliable probabilities.

[11] In this step, I will shift from PySpark to the native Python ecosystem (particularly pandas and sklearn). The reason is that PySpark is undeniably the gold standard for distributed data engineering and it flawlessly handled the initial raw data extraction and stratified splitting without hitting any memory bottlenecks. However, utilizing Spark MLlib for the entire hyperparameter tuning phase would have created a massive performance bottleneck due to JVM communication overhead across thousands of trials. To prevent data leakage during the 3-fold cross-validation, I intentionally moved the preprocessing steps out of Spark and integrated them directly into Scikit-Learn Pipeline objects. This ensures that scaling and encoding statistics are calculated strictly on the training folds. The reason for using 3 folds in cross-validation is that I am dealing with a relatively large dataset. Using more folds would not provide significant benefits due to the law of large numbers but would significantly increase training time.

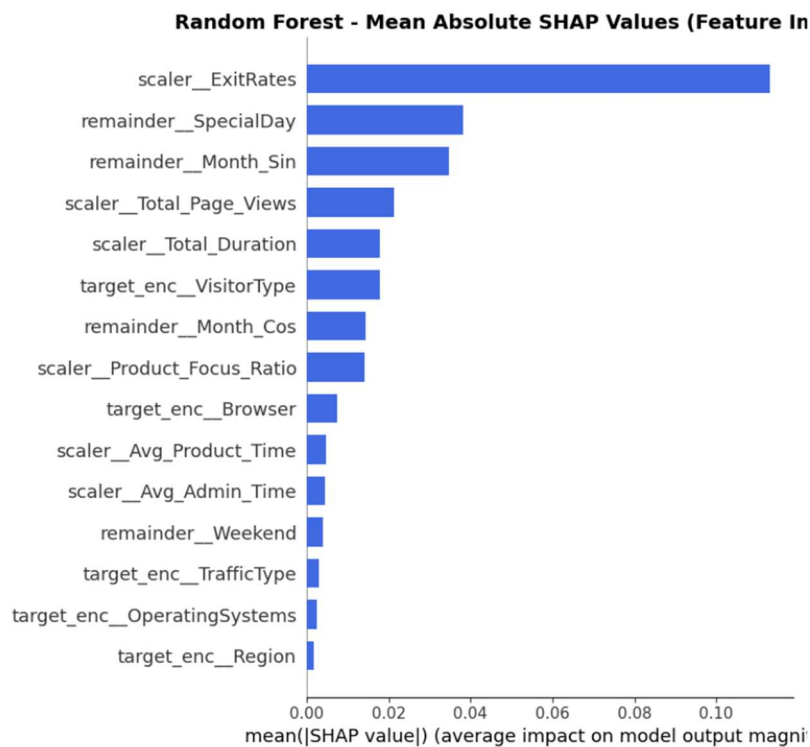
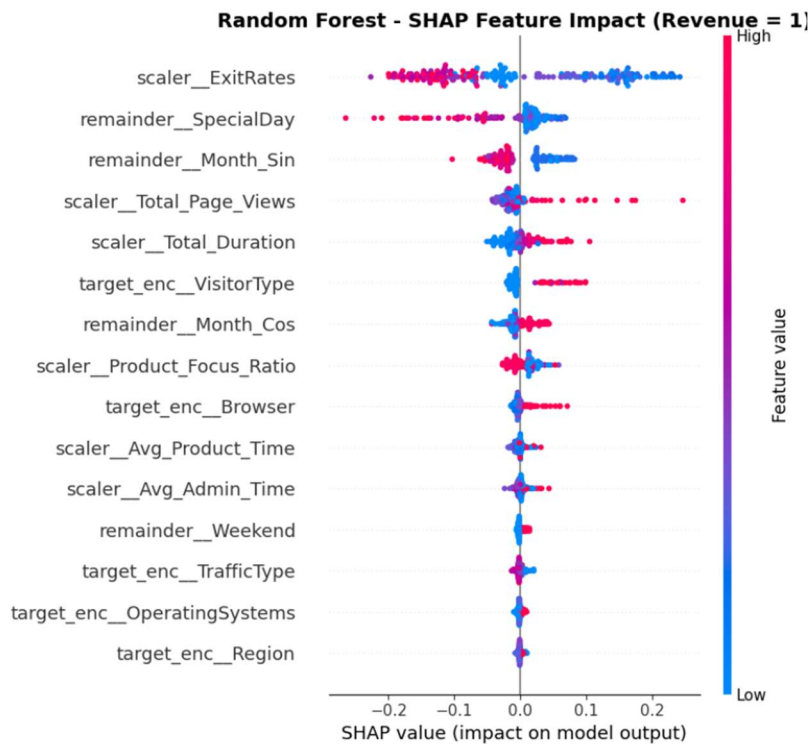
A More Sophisticated Approach

[12] My foundational models (LightGBM, XGBoost, and Random Forest) already utilize sequential boosting and parallel bagging. Attempting to apply yet another layer of bagging or boosting on top of these models would introduce massive computational latency with almost

no gain in predictive power. Stacking, however, operates in a different way where it leverages the algorithmic diversity of heterogeneous models. The Stacked Ensemble acts as a board of directors by taking the distinct approach of each algorithm to form a consensus rather than forcing them to go through the same operations again. But why logistic regression as a meta-learner? Complex meta-learners tend to aggressively memorize predictions provided by the base estimators, which leads to overfitting. Thus, it strictly evaluates which base model is most trustworthy in specific probability ranges without overcomplicating the final decision boundary. By passing tree-based learners' raw predictions through a linear meta-learner, it is expected to see the Stacked Ensemble smooth out these overconfident spikes. The final version of my stacked ensemble looks like:

```
StackingClassifier(cv=3,
                  estimators=[('lightgbm',
                              Pipeline(steps=[('preprocessor',
                                              ColumnTransformer(remainder='passthrough',
                                                                transformers=[('target_enc',
                                                                    TargetEncoder(random_state=616),
                                                                    ['TrafficType',
                                                                     'VisitorType',
                                                                     'OperatingSystems',
                                                                     'Browser',
                                                                     'Region'])),
                                              ('scaler',
                                               RobustScaler(),
                                               ['Avg_Product_Time',
                                                'Avg_Admin_Time',
                                                'Total_Duration',
                                                'Total_Page_Views',
                                                '...',
                                                ['Avg_Product_Time',
                                                 'Avg_Admin_Time',
                                                 'Total_Duration',
                                                 'Total_Page_Views',
                                                 'ExitRates',
                                                 'Product_Focus_Ratio']]]))),
                              ('clf',
                               RandomForestClassifier(max_depth=20,
                                                       max_samples=0.47198373709196506,
                                                       min_samples_leaf=4,
                                                       n_estimators=900,
                                                       n_jobs=1,
                                                       random_state=616)))]),
                  final_estimator=LogisticRegression(C=4.466542759222052,
                                                       max_iter=1000, n_jobs=1,
                                                       random_state=616),
                  n_jobs=-1)
```

[13] Random Forest can be used to visualize feature importance in model predictions:



The bar plot tells how important a feature is in general. The beeswarm plot shows the direction and magnitude of that feature's influence on each individual customer.

It is clear that ExitRates is the most important feature the model uses as the bar plot indicates. There is a tight cluster of blue dots (low ExitRates) far to the right, which signifies a very high positive impact on purchase probability. On the other hand, pink/red dots (high ExitRates) are

heavily clustered on the left (negative SHAP values). These results simply indicate that customers who visit pages that are frequently the final page in a session are much less likely to buy. Minimal ExitRates are the strongest indicator of intent.

SpecialDay emerges as the second strongest feature with its low values showing a positive impact on purchase probability. Month_Sin occupies the third position. It shows a strong negative relationship in the beeswarm plot: high Month_Sin values (often associated with late spring or summer months in a sinusoidal transform) decrease the likelihood of purchase (large blue cluster on the right, pink on the left). Low Month_Sin values, which typically correspond to the winter/holiday shopping season, have a positive impact. Seasonal cyclicalities are a major driver of behavior.

Total_Page_Views and Total_Duration occupy positions fourth and fifth. In both beeswarm plots, pink/red dots (high values) spread far to the right, which means that they show a clear positive correlation with purchase probability. The blue clusters on the other hand are dense around zero or slightly negative. Viewing more pages and staying longer can be used as a reliable predictor of intent.

For VisitorType_TE, it is seen a distinct pink cluster centered on the positive SHAP axis. High values here (associated with the "Returning Visitor" category from the target encoding) have a predictable positive impact on likelihood whereas low values ("New Visitor") do not.

Product_Focus_Ratio shows a negative relationship. Blue dots (low ratio values) are on the right, while pink dots (high values) are on the left. A low "Product Focus Ratio" in engineered feature means a higher percentage of time spent on product-related pages versus other types. This means that customers focused primarily on products have higher intent.

And for the variable Weekend, it is seen a distinct pink cluster on the positive side of zero. The probability of a customer purchasing increases on the weekend.

Both plots confirm that features like Avg_Product_Time and Avg_Admin_Time have very little impact on the model's final output. The dots are all tightly clustered around the zero line and they rank near the bottom of the importance chart. Spending time on an individual product page or administrative page does not matter nearly as much as the total engagement and aggregate views.

[14] In a real world e-commerce environment, taking action based on raw model outputs without quantifying uncertainty is a massive risk. I am therefore going to use a conformalization framework to overcome this issue. Thus, I will ensure a guaranteed confidence interval for probability outcomes for any given error tolerance (such as 90% or 95% confidence). If the model is highly certain, it outputs a single class [1] but if the customer's behavior is borderline, it outputs a set of both classes [0, 1], which means the prediction is uncertain. This acts as the ultimate risk management shield. It is thus possible to allow stakeholders to easily identify which predictions are solid and which require a cautious approach.

Evaluation Based on Relevant Metrics

[15] While the calibration step is very useful for risk analysis, some business operations (like automated email campaigns or targeted ads) require binary triggers. An automated system cannot execute a marketing campaign based on a $[0, 1]$ output. It needs a definitive 1 or 0. Therefore I still need to find an optimal specific probability threshold triggers a "Purchase Predicted" flag. I am going to optimize the decision threshold specifically for the F2 Score.

Traditional metrics like Log Loss or the Brier Score evaluate the smoothness of the probability curve but not the profitability of the final "yes" or "no" decision the marketing team intends to get. To maximize revenue, it is necessary to prioritize capturing every potential buyer (high recall) while maintaining a reasonable level of accuracy (precision). If the model is overly optimistic and the marketing team accidentally sends an email coupon to a user who was never going to buy (false positive), the cost is zero. But if the marketing team fails to send a coupon to a high-intent customer who just needed a slight nudge (false negative), an entire sale is lost. The model must prioritize capturing as many potential buyers as possible (high recall) without completely losing its exactness (precision) to maximize company revenue. F2 Score is important at this point in that the standard F1 Score treats precision and recall equally but the F2 Score weights recall twice as heavily.

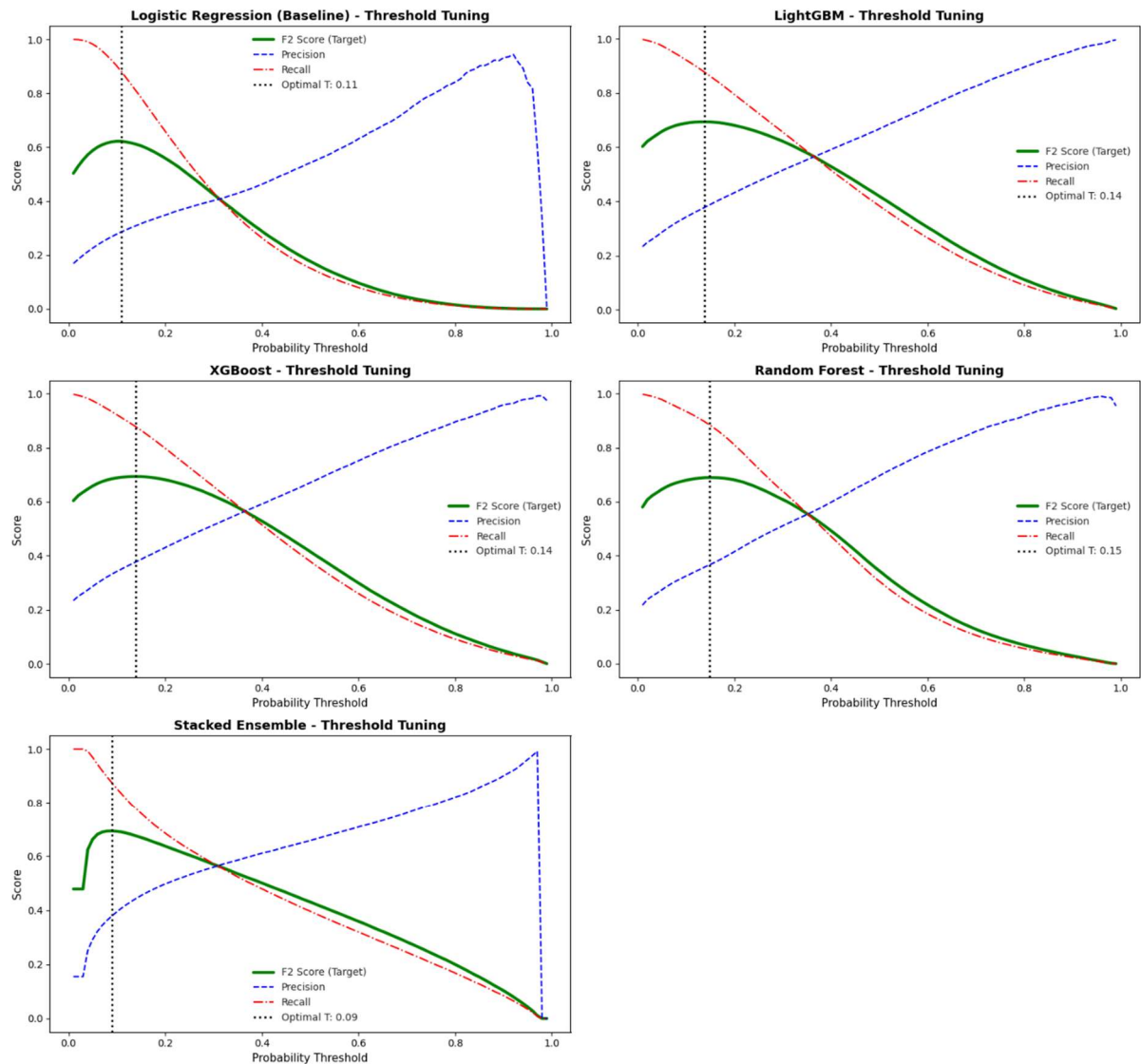
Then why did not I optimize the F2-Score instead of the Brier Score during hyperparameter tuning? To the F2-Score a customer with a 51% probability of buying looks exactly the same as a customer with a 99% probability. If the algorithm is forced to train based on this metric, the model stops learning about customer behavior. It just focuses on crossing an arbitrary finish line instead. The Brier Score on the other hand is a continuous probabilistic metric and it penalizes overconfidence. I force the model to be honest and accurately reflect the true probability of the real world by optimizing the Brier Score first. It is possible to tune the threshold using out-of-fold predictions via cross validation. I use these out-of-fold predictions to compare model performance in what follows. The results are:

```
Out-of-Fold (OOF) F2 Scores
Logistic Regression (Baseline): 0.6220
LightGBM: 0.6936
XGBoost: 0.6933
Random Forest: 0.6895
Stacked Ensemble: 0.6934
```

LightGBM achieves the highest F2 Score with a value of 0.6936. It is followed by Stacked Ensemble and XGBoost with marginal differences (0.6934 and 0.6933 respectively). I believe that the reason why the stacked ensemble could not achieve the highest score is that there is not much diversity among its base learners. Since XGBoost, LightGBM, and Random Forest are all tree-based models, they lack diversity. Better performance may be achieved by combining algorithms that use different prediction methods. The worst-performing algorithm is logistic regression with an F2 score of 0.6220, which represents a considerable difference compared

to the other algorithms. This is expected as it is the simplest model among them and took only about 15 minutes of tuning.

Roughly speaking, all algorithms achieved their highest F2 scores at decision thresholds between 0.10-0.18:



[16] After threshold-tuning, I apply my calibration logic on each of the algorithms:

```

Calibrator(estimator=Pipeline(steps=[('preprocessor',
                                     ColumnTransformer(remainder='passthrough',
                                                         transformers=[('target_enc',
                                                                           TargetEncoder(random_state=616),
                                                                           ['TrafficType',
                                                                           'VisitorType',
                                                                           'OperatingSystems',
                                                                           'Browser',
                                                                           'Region'])],
                                                         ('scaler',
                                                          RobustScaler(),
                                                          ['Avg_Product_Time',
                                                          'Avg_Admin_Time',
                                                          'Total_Duration',
                                                          'Total_Page_Views',
                                                          'ExitRates',
                                                          'Product_Focus_Ratio'])])),
                                ('clf',
                                 LogisticRegression(C=4.466542759222052,
                                                    max_iter=1000,
                                                    n_jobs=1,
                                                    random_state=616))]),
          optimal_threshold=np.float64(0.10898989898989898))
Calibrator(estimator=Pipeline(steps=[('preprocessor',
                                     ColumnTransformer(remainder='passthrough',
                                                         transformers=[('target_enc',
                                                                           TargetEncoder(random_state=616),
                                                                           ['TrafficType',
                                                                           'VisitorType',
                                                                           'OperatingSystems',
                                                                           'Browser',
                                                                           'Region'])],
                                                         ('scaler',
                                                          RobustScaler(),
                                                          ['Avg_Product_Time',
                                                          'Avg_Admin_Time',
                                                          'Total_Duration',
                                                          'Total_Page_Views',
                                                          'ExitRates',
                                                          'Product_Focus_Ratio'])])),
                                ('clf',
                                 LogisticRegression(C=4.466542759222052,
                                                    max_iter=1000,
                                                    n_jobs=1,
                                                    random_state=616))]),
          optimal_threshold=np.float64(0.10898989898989898))
...
n_jobs=-1),
optimal_threshold=np.float64(0.08919191919191918))

```

[17] What if I want to squish every operation I did so far into a single pipeline? Think of this pipeline as a highly automated assembly line. At the one end, we feed it completely raw customer data straight from the database. Once the data enters the pipeline, it flows through the entire system without any human intervention. The pipeline automatically cleans the data, executes feature engineering by removing irrelevant columns and creating new features using the interaction logic discussed earlier, applies the optimal decision threshold, makes predictions using the tuned model, and outputs a decision table for the marketing team. I specifically create a single end-to-end ML pipeline using the best-performing model (LightGBM).

Interpretation of Results

[18] I previously used out-of-fold F2 scores to compare the models and select the best one. But how does the best-performing model perform on the test set? I use my end-to-end pipeline to make predictions on the test set:

Log_Loss	: 0.2732
Brier_Score	: 0.0868
Balanced_Accuracy	: 0.8055
F2_Score	: 0.6901
Recall	: 0.8727
Precision	: 0.3757

I am going to report Log Loss and Brier Score to prove my probabilities are trustworthy, Balanced Accuracy to prove the model is not cheating the imbalanced data, and the Recall/Precision/F2 Scores to prove the model is suitable to use for the business objective.

Recall (0.8727): This means it successfully identified 87.27% of all actual buyers.

Precision (0.3757): This is the cost of high recall. A precision of 37.5% means that when the model flags a user as a “buyer,” it is correct about 1 out of 3 times. I report this to be transparent about the trade-off, which is that I accept a higher number of false positives (marketing to someone who will not buy) to ensure capturing those who will.

Balanced Accuracy (0.8055): If a website has a 90% non-buyer rate, an incompetent model could just predict "no one will buy anything" and achieve 90% normal accuracy. Balanced Accuracy calculates how well the model predicts both the majority class and the minority class and averages them. An 80.5% here is an acceptable outcome given that I did not optimize for this metric.

Brier Score (0.0868): Think of this as the "mean squared error" for probabilities. A score this close to 0 proves that when the model says there is a 10% chance, the reality is very close to 10%.

Log Loss (0.2732): This metric heavily penalizes a model for being confident but wrong. A low log loss confirms that the model is properly calibrated.

[19] I am going to directly interpret a row of the analyzed dataset returned by the end-to-end pipeline now:

Revenue	Raw_Probability	Tuned_Prediction	CP_99_Set	CP_95_Set	CP_90_Set	CP_80_Set
0	0.36162388	1	[0, 1]	[0, 1]	[0]	[]

The number 0 under Revenue means this customer actually did not end up making a purchase. This is what happened in reality. Yet the Tuned_Prediction is 1, which means the standard model made a mistake and falsely predicted a sale. With a Raw_Probability of roughly 0.36 or 36%, the model saw enough positive signals to cross its decision threshold and label this person a buyer, but it was clearly far from certain.

This hidden doubt is exactly what the CP sets reveal and why they are so valuable. While the standard prediction blindly guaranteed a sale, the CP sets catch the hesitation. When 99% or 95% certainty is demanded, the model admits it cannot guarantee a purchase, outputting a [0, 1] set to show the person could easily go either way. Interestingly, when the requirement is

lowered to 90%, the set shifts to $[0]$, and at 80%, it yields an empty set. This mathematical instability perfectly illustrates a customer who is completely on the fence and highly unpredictable.

Knowing all these gives the marketing team a critical advantage. If it is only relied on the Tuned_Prediction, it would have assumed that this customer was a guaranteed buyer, offered them no incentives, and lost the sale (just as it happened in reality). Because this shopper is a true “persuadable”, this is the exact scenario where deploying a strong 15% or 20% discount is the perfect approach to actively change their mind and push them toward a purchase.

All in all, the most profitable strategy here is to use CP to catch these uncertain customers before they leave. By offering a substantial discount to users where the CP set flags deep hesitation, we can convert lost opportunities into actual revenue without wasting budget on those who are already going to buy.